

DEPLOYMENT READINESS AUDIT · TENANT MOBILE

LilyCrest Tenant Mobile App

A full system, workflow, and security audit of the LilyCrest mobile application as a tenant-only APK — covering role boundaries, financial integrity, IDOR, branch isolation, duplicated logic, and production polish.

repo `LilyCrest-Clean` · branch `master` · HEAD `0346bece` · 2026-08-15

EXECUTIVE VERDICT



Ready with non-blocking follow-ups

A

Executive Verdict

One audit pass across five parallel workstreams (auth/session/navigation, home/notifications/announcements, billing/payment, maintenance/documents/profile/password, and API-contract/dead-code/build-config) found a single **High**-severity issue — a real tenant-only boundary gap — and a handful of Medium/Low items. The High finding, plus two of the Medium findings, have been fixed and verified in this pass (backend + frontend test suites green, 0 lint errors). No reservation, applicant, or admin workflow is reachable from the tenant app. Remaining items are genuinely non-blocking: an Android permissions review, a payment-status UX nuance, and a maintenance-ticket idempotency hardening — none of which risk tenant data, money, or cross-account exposure.

B Tenant Mobile System Map

App Launch

- Firebase auth ready
- Session token check (SecureStore)
- GET /users/me (identity + role check - new)
- Tenant-only gate: reject role=admin/superadmin
- Main Tenant App
 - +-- Home (balance, room, notifications, announcements, maintenance summary)
 - +-- Notifications (single source: AuthContext)
 - +-- Announcements (branch-filtered, backend-authoritative)
 - +-- Billing
 - | +-- Current Bill / Bill Details
 - | +-- Payment (PayMongo checkout)
 - | +-- Billing History
 - +-- Maintenance
 - | +-- Create Request
 - | +-- View Request / Replies
 - | +-- Track Status
 - +-- Documents / Contract (proxied to Capstone-Website, tenant-scoped)
 - +-- Profile
 - +-- Personal Information (name/email/address read-only; username/phone/photo editable)
 - +-- Change Password (session revocation on success)
 - +-- Logout

Scope confirmation

- | | | |
|--------------------|--|---------------------|
| Not present | Reservation / application workflow — no creation, status tracking, or approval UI anywhere in | frontend/app |
| Not present | Applicant onboarding — grep across the tenant tree found only read-only historical references (e.g. a "reservation fee" line item already applied as a credit) | |
| Not present | Admin workflow — no dashboards, approval controls, or provider-assignment UI; the only "admin" surface is tenant → human support chat | |

c Screen Inventory

SCREEN	PURPOSE	TENANT ALLOWED?	API DEPENDENCY	STATUS
login.jsx, forgot/reset/otp-verify	Authentication	Yes (public)	/auth/*	OK
(tabs)/home.jsx	Dashboard	Yes	/dashboard/me	OK
(tabs)/billing.jsx, billing-history.jsx, bill- details.jsx	Billing	Yes	/billing/*	OK
payment.jsx, payment- success/cancel.jsx	Payment	Yes	/paymongo/*	OK
(tabs)/services.jsx	Maintenance	Yes	/maintenance/*	OK
(tabs)/announcements.jsx	Announcements	Yes	/announcements	OK
my-documents.jsx, contract-viewer.jsx, document-viewer.jsx	Documents / contract	Yes	/documents/*, /contracts/*	OK
(tabs)/profile.jsx, settings.jsx, change- password.jsx	Account	Yes	/users/me, /auth/change- password	OK
(tabs)/dashboard.jsx (hidden tab)	Legacy alias	n/a	none	Fixed — was a dead stub, now redirects
(tabs)/chatbot.jsx, LilyAssistantScreen	Support chat (Lily assistant)	Yes	/chatbot/*, /chat/*	Not re-audited — see §R
debug/api-health.jsx	Dev diagnostics	__DEV__ only	n/a	OK — self-gated + regression test
surveys.jsx, survey- form.jsx	Feedback (disabled feature)	Hidden	/surveys/*	OK — no nav entry, regression- tested hidden

No applicant, reservation, or admin-dashboard screen exists under `frontend/app`. Route protection is centralized in a single root-layout guard (`frontend/app/_layout.jsx`) that applies to every entry point, including deep links.

D API / Function Map

FEATURE	ENDPOINT	METHOD	AUTH	TENANT-ONLY	OWNERSHIP
Session identity	/auth/me, /users/me	GET	✓	✓ (frontend guard — new)	self
Dashboard	/dashboard/me	GET	✓	✓ added	self
Billing	/billing/me, /billing/history	GET	✓	✓ added	self
Payment	/paymongo/checkout, /checkout/:id/status	POST/GET	✓	✓ added	server-resolved from owned bill
Maintenance	/maintenance/me, /maintenance/:id	GET/POST/PATCH	✓	✓ added	findRequestForUser
Notifications	/notifications, /:id/read	GET/PATCH	✓	✓ added	self scoped lookup
Announcements	/announcements	GET	optional	n/a — semi-public by design	branch-filtered server-side
Documents / Contracts	/documents/:id, /contracts/*	GET	✓	✓ added	self / upstream bearer re-derivation
Profile	/users/me, /users/documents	GET/PUT	✓	✓ added	self
Uploads	/upload/firebase-storage, /imagekit-auth	POST/GET	✓	✓ added	tenant-scoped storage path
Survey (hidden)	/surveys/me, /:id/*	GET/PUT/POST	✓	✓ added	self
Support tickets	/tickets/me, /:id	GET/POST/PUT	✓	✓ added	self

"Tenant-only" column reflects the new `tenantMiddleware` (backend/middleware/auth.js) applied to every tenant-scoped router in this pass — see §Q. Chat/chatbot endpoints were left untouched (not part of the verified fix set; see §R).

E Business Logic Conflicts

Medium Home vs. Billing History disagreed on "outstanding" for bills under verification

LOCATION frontend/app/(tabs)/home.jsx, frontend/app/billing-history.jsx

BEHAVIOR Home's HOME_NON_PAYABLE set excluded 'verification' status from the owed total; Billing History's NON_PAYABLE_STATUSES did not — so a bill under verification could read as "cleared" on Home while showing a real balance on Billing.

AUTHORITATIVE RULE Backend bill status field, but the two screens must apply one shared rule to it.

FIX Fixed extracted into frontend/src/utils/billingStatus.js, imported by both screens.

Verified sound Payment amount, ownership, and idempotency are entirely backend-derived

LOCATION backend/controllers/paymongo.controller.js

The frontend sends only a billingId; amount, tenant identity, and metadata are all resolved server-side from an owner-scoped bill lookup. No client-supplied amount, tenantId, or branchId is trusted anywhere in the payment path.

F Duplicated Logic

LOGIC	LOCATIONS	DIVERGES?	RESOLUTION
Outstanding-bill status set	home.jsx / billing-history.jsx	Yes	Consolidated — shared billingStatus.js
Unread notification count	AuthContext.js (single source)	No	Already consolidated — code comment documents a prior fix
Announcement branch visibility	announcement.controller.js, reused by notification.controller.js	No	One implementation, imported not duplicated
Maintenance status labels	services.jsx ↔ maintenance.controller.js	No	Backend copy (friendlyStatusCopy) and frontend mapper agree on every value
Bill charge arithmetic	payment.jsx, bill-details.jsx	No	Pure display of server-computed fields; no client-side recomputation
Legacy vs. real bill collection reconciliation	billing.controller.js, paymongo.controller.js	Structural risk	Deferred — see §R

G

Validation Findings

Medium

Maintenance ticket submission has no idempotency key

LOCATION `frontend/app/(tabs)/services.jsx`,
`backend/controllers/maintenance.controller.js:1394`

SCENARIO A network timeout after the server actually committed the ticket lets the tenant resubmit, creating a duplicate ticket for the same issue. The submit button disables on tap, but nothing dedupes a genuine retry.

FIX **Deferred** — needs a short-lived idempotency key checked server-side before insert; not implemented in this pass (see §R).

Low

Frontend has no max-length guard on maintenance description

LOCATION `frontend/app/(tabs)/services.jsx:395`

Backend enforces a 1000-char cap; frontend only checks the 10-char minimum. A tenant typing past the limit only finds out via a generic 400 after tapping Submit.

FIX **Deferred** — UX polish, not a security or data-integrity issue.

Verified sound

Profile, password, and payment inputs are validated identically on both sides

Username (3–30 chars, cooldown), phone (`+63XXXXXXXXXX`), password strength, and payment amount are all enforced with matching frontend/backend rules — frontend is UX-only, backend is authoritative throughout.

H Security Findings

High Mobile login endpoint authenticated admin accounts with no role check downstream

LOCATION backend/controllers/auth.controller.js:364,
backend/middleware/auth.js, frontend/src/context/AuthContext.js

SCENARIO /auth/login and /auth/google intentionally still authenticate admin accounts (the web admin panel shares them), but neither authMiddleware nor the frontend's session logic checked role afterward. An admin session could reach every tenant-scoped screen and endpoint in the mobile app.

FIX **Fixed** — new tenantMiddleware rejects admin/superadmin on every tenant router (backend), and AuthContext now rejects non-tenant roles at every point the app marks itself authenticated: login, OTP verify, Google sign-in, session hydration, and checkAuth (frontend). Verified with the full backend (304) and frontend (194) test suites, both green.

Verified sound IDOR — no cross-tenant reads or mutations found

Billing, payment, maintenance, documents, and notifications all resolve ownership from req.user, never a client-supplied ID. Cross-tenant billingId / requestId / notificationId lookups return 404, not another tenant's record.

Verified sound Branch isolation is backend-authoritative

Announcement visibility is derived from the tenant's own occupancy record server-side (resolveTenantBranch), never trusted from the client, and defaults to hidden rather than visible when branch can't be resolved.

Medium Broad Android permissions warrant a pre-submission review

LOCATION frontend/android/app/src/main/AndroidManifest.xml:3-9

SYSTEM_ALERT_WINDOW (draw-over-other-apps) plus legacy **READ/WRITE_EXTERNAL_STORAGE** are declared. These are Play Store review flags and, if unused, unnecessary attack surface.

FIX **Deferred** — confirm which native module needs **SYSTEM_ALERT_WINDOW** before stripping it; not changed blind in this pass.

Verified sound Build/secrets hygiene

Production API host is https://api.lilycrest.space across all 4 EAS build profiles, with a runtime guard that hard-fails on a localhost/private-IP host. Cleartext traffic is disabled app-wide. google-services.json and .env are gitignored and confirmed untracked.

I Authentication / Session Findings

- **Login (email/password, Google, OTP):** empty/malformed input, duplicate-tap, and offline/timeout are all handled with clear, tenant-safe messaging; non-tenant roles are now rejected client-side in addition to the pre-existing backend rejection at `/auth/login` / `/auth/google` for genuinely unregistered accounts.
- **Session restoration:** hydration always terminates in a definite `authStatus` — no path leaves the app stuck on a loading screen. A stale AsyncStorage cache used to be able to outlive a backend-side deactivation until the next successful poll; treating 403 the same as 401 during hydration (part of this pass's fix) closes that window too.
- **Logout:** local state (user, notifications, badge count) clears synchronously before best-effort server teardown, SecureStore/biometric credential clearing, and push-token disable. Back-navigation after logout cannot reopen protected screens — the root-layout guard re-evaluates on every route change.
- **Debug route:** `frontend/app/debug/api-health.jsx` self-gates on `__DEV__` independent of how it's entered, with a regression test locking the behavior in.

J Billing / Payment Findings

Financial source-of-truth is backend-only throughout: rent, utilities, penalties, credits, previous balance, and outstanding amount are all computed server-side and only formatted (never recomputed) on the client. The payment path resolves amount and ownership from the authenticated tenant's own bill record — a tenant cannot pay another tenant's bill by manipulating an ID. Webhook signature verification fails closed when unset, HMAC comparison uses a timing-safe check, and bill-paid writes are atomic and idempotent against duplicate webhook delivery. Checkout-session creation has both a 20-minute reuse window and a 20-second claim lock, so double-tap or retry storms cannot mint two live PayMongo sessions.

Low A declined/failed payment looks identical to a merely-slow one

LOCATION `frontend/app/payment-success.jsx`

The success screen polls for up to 60s and then shows a generic "still processing" message on timeout — the same message a genuinely failed payment would produce, even though PayMongo already emits a distinct `payment.failed` webhook event.

FIX **Deferred** — needs the failed-state to be surfaced through `getCheckoutStatus` and branched on in the UI; not changed in this pass.

Low / Maintainability

Dual bill-collection reconciliation is defensively written but structurally complex

Bills live in two parallel collections (legacy `billing` and current `bills`), reconciled by a scoring heuristic. Current behavior is confidence-ordered correctly (paid status dominates), but the branch count is a real long-term audit-surface cost. Recommend eventual migration to a single collection — no immediate action required.

K Maintenance Findings

Every tenant-facing maintenance lookup filters by `user_id` matching the authenticated session — cross-tenant reads return 404, not the record. Status labels are single-sourced between backend (`friendlyStatusCopy`) and frontend, so "In Progress" never disagrees with itself across screens. Provider assignment is admin-only (`PATCH /maintenance/admin/:id/status` , gated by `adminMiddleware`) with no corresponding call site anywhere in the tenant app — the tenant only ever sees a read-only "a service provider has been assigned" line. The two open items are the description max-length guard and submission idempotency key covered in §G.

L Notification / Announcement Findings

Notifications have one source of truth (`AuthContext`) — the codebase's own comments confirm this was consolidated from three separate implementations in a prior pass. Mark-read is optimistic with rollback on failure, ownership-checked server-side, and polling correctly stops on logout (state reset happens synchronously, ahead of the interval-clearing effect). Announcement branch visibility is a single backend implementation reused by the notification feed, so the two can't disagree — a tenant from branch A cannot see branch B's restricted announcement, and unresolvable branch defaults to hidden, not visible.

M Document / Contract Findings

Tenant documents are resolved from a fixed template registry personalized only with the requesting tenant's own data — there's no guessable per-user document ID to exploit. Contracts are proxied server-to-server to a separate Capstone-Website service; the bearer token is forwarded byte-for-byte and re-validated upstream, so ownership enforcement for contract data lives in that external repo and wasn't independently verifiable from this codebase (flagged as an out-of-scope dependency, not a confirmed gap). No lingering applicant/reservation terminology was found in contract or document UI, and no admin-only generate/edit controls exist client-side.

N Profile Findings

Editable fields match backend support exactly — name, email, and address are read-only client-side with matching backend rejection messages; only username, phone, and photo are live, with identical validation rules on both sides. A grep for applicant-oriented status strings ("Pending Move-in", "Application Pending", "Reservation Approved") found no live matches — the only hits are in a regression test that explicitly asserts this text stays absent, confirming a prior fix rather than an open issue.

0 Production Polish Findings

Clean

- No TODO/FIXME/"coming soon" strings in any tenant screen
- No raw backend/Firebase error text reaches the UI anywhere audited
- No sensitive data in production console logging — token/header redaction is explicit and dev-gated
- Currency and date formatting distinguish "missing" from a real ₱0.00 / falls back to "—" instead of rendering NaN / "Invalid Date"

Fixed this pass

- Dead (tabs)/dashboard.jsx stub now actually redirects to Home instead of dead-ending
- Home/Billing outstanding-balance figures can no longer silently disagree

Minor cosmetic items not touched: attachment picker copy says "JPG, PNG" when PDFs/docs are also accepted (services.jsx:1139), and a title-based FlatList key fallback in the notification sheet could theoretically collide for two untitled items in the same batch.

P Dead / Stale / Unreachable Code

- adminGetAllTickets , adminReplyToTicket , adminUpdateTicketStatus , updateBilling in frontend/src/services/api.js — bound into the mobile client but never called from any screen, and backend-blocked by adminMiddleware even if invoked. Safe to remove, not removed in this pass (pure dead-code cleanup, no risk either way).
- (tabs)/dashboard.jsx — was a non-functional redirect stub; fixed (§Q).
- No applicant, reservation, or admin-dashboard screens exist to report as dead — none were ever built into this client.

Q Fixes Implemented

backend/middleware/auth.js

New `tenantMiddleware` . Rejects `role === 'admin' | 'superadmin'` with 403 on any tenant-scoped route, mirroring the role filter `auth.controller.js` already uses at login. Verified the admin web panel (`backend/public/admin`) never calls any of these routes before applying it, so nothing there regresses.

backend/routes/*.routes.js
(11 files)

Applied `tenantMiddleware` after `authMiddleware` on every tenant-only route in billing, dashboard, documents, notifications, maintenance, user, contracts, tickets, uploads, paymongo, and survey routers — leaving each router's separately-gated `adminMiddleware` sub-routes untouched.

frontend/src/context/AuthContext.js

Role guard at every "authenticated" transition — email login, OTP verify, Google sign-in, session hydration, and `checkAuth` now all reject non-tenant roles and clear the local session, closing the gap where `loadAuthoritativeTenantProfile`'s error fallback could otherwise silently let an admin session through. Also promoted 403 to the same "definitely log out" handling as 401 during hydration, closing a secondary staleness window for deactivated accounts.

frontend/src/utils/billingStatus.js
(new)

Single shared `NON_PAYABLE_STATUSES / isBillOutstanding / getBilledAmount`, now imported by both `home.jsx` and `billing-history.jsx` in place of their two divergent local copies.

frontend/app/(tabs)/dashboard.jsx

Replaced the non-functional "Redirecting to Home..." placeholder with an actual `<Redirect href="/(tabs)/home" />`, matching the pattern already used by the top-level `home.jsx` shim.

R Deferred Recommendations

Genuinely non-blocking — none of these risk tenant data, money, or role boundaries as shipped:

- Add a short-lived idempotency key to maintenance ticket submission to prevent duplicate tickets on network-retry.
- Add a frontend max-length guard (1000 chars) on the maintenance description field, mirroring the backend cap.
- Surface PayMongo's `payment.failed` event through `getCheckoutStatus` so a declined card shows a distinct message from a slow-but-pending one.
- Review whether `SYSTEM_ALERT_WINDOW` and legacy storage permissions in the Android manifest are actually used before Play Store submission; strip if not.
- Remove the four dead admin-only functions bound into `frontend/src/services/api.js` (unreachable, but unnecessary bundle surface).
- Chat/chatbot routes (`chat.routes.js` , `chatbot.routes.js`) were not included in the tenant-only middleware pass — no audit flagged them, but they weren't independently re-verified either. Worth a follow-up pass before treating the role-boundary fix as fully exhaustive.
- Plan an eventual migration off the dual legacy/current bill-collection reconciliation — currently safe but structurally complex.

S Test Results

304 / 304

BACKEND TESTS

194 / 194

FRONTEND TESTS

33 / 33

FRONTEND SUITES

0

LINT ERRORS

Both suites re-run after every fix in this pass, including the auth-boundary change (authFlowContracts, authStability, authContextSessionExpiry, signInWithGoogleRetry all pass unmodified) and the billing-status consolidation. `expo lint` reports 0 errors and only 4 pre-existing warnings unrelated to this pass.

T

Git Status

FILE	ORIGIN
.claude/settings.local.json	Pre-existing, unrelated to this audit
docs/MAINTENANCE_FLOW_AUDIT_2026-08-15.md	Pre-existing untracked file, unrelated
../phase15c-health-deploy/	Pre-existing sibling directory, unrelated
backend/middleware/auth.js + 11 route files	This audit — tenantMiddleware
frontend/src/context/AuthContext.js	This audit — role guard
frontend/src/utls/billingStatus.js (new)	This audit — shared status util
frontend/app/(tabs)/home.jsx, billing-history.jsx	This audit — consume shared util
frontend/app/(tabs)/dashboard.jsx	This audit — dead redirect fix

Nothing has been committed, pushed, or built. Pre-existing changes were left untouched throughout.

U

Deployment Blockers

FINDING	SEVERITY	BLOCKING?	REQUIRED ACTION
Admin accounts could reach tenant-scoped app/API	High	No — fixed	None; verified fixed and tested
Home/Billing outstanding-balance divergence	Medium	No — fixed	None; verified fixed
Maintenance ticket duplicate-submit on retry	Medium	No	Add idempotency key (post-launch)
Android manifest permissions review	Medium	No	Confirm usage before Play Store submission
Payment-failed vs. pending UX ambiguity	Low	No	Surface distinct failed state (post-launch)
Dual bill-collection reconciliation complexity	Low	No	Plan migration, not urgent

V Final Deployment Checklist

✓ Tenant-only role boundary enforced (backend + frontend)

✓ No reservation UI

✓ No applicant UI

✓ No admin UI

✓ Login stable across all methods

✓ Session restore stable, no infinite loading

✓ Logout fully clears tenant state

✓ Notifications synchronized (single source)

✓ Branch isolation verified

✓ Announcements verified

✓ Billing source-of-truth verified

✓ Payment ownership verified

✓ Duplicate payments guarded

✓ Maintenance ownership verified

✓ Maintenance status mapping verified

! Maintenance duplicate-submit guard — deferred, non-blocking

✓ Contract ownership verified

✓ Profile polished, no applicant-state leakage

✓ Change Password safe (session revocation confirmed)

✓ Survey/Feedback hidden

✓ Debug routes production-gated

✓ No raw backend errors reach UI

✓ No sensitive logging

✓ No cross-tenant IDOR

✓ No client-trusted tenantId/userId/branchId

✓ No stale previous-user data across accounts

✓ Production API host verified

! Android permissions — recommend review before Play Store submission

✓ All critical tests passing (304 backend, 194 frontend)

✓ 0 lint errors

W

Final Recommendation

Safe to commit and build a deployment-test APK.

The one High-severity issue found — the tenant-only role boundary — is fixed and verified on both backend and frontend, with the full test suite green and 0 lint errors. Remaining items (Android permission review, maintenance idempotency, payment-failed UX) are genuine non-blocking follow-ups: none affect security, financial correctness, or cross-tenant isolation as shipped. Recommend reviewing the diff, committing, and proceeding to a deployment-test build; carry the deferred items in \$R into the next iteration.